EPIC-A — Registration, Identity Verification & Admin Review Queue — Technical Spec

Status: Draft — for stakeholder review **Date**: 14 July 2026 **Author**: Adrian Hall (Technical Lead), drafted with Claude Code **Scope**: The first per-epic technical spec, building on the approved cross-cutting architecture in `docs/superpowers/specs/2026-07-14-askapeer-architecture-design.md` ("the architecture spec"). This document does not repeat that spec's stack, hosting, or schema rationale — it only adds detail specific to EPIC-A. Read the architecture spec first, particularly Sections 4 (`identity` schema) and 5.1–5.2 (verification pipeline, authentication).

Source of truth for product requirements: `docs/askapeer-prd-v0.1.md`, Section 8 (Verification & Trust Model).

EPIC-A is the foundation epic: no other epic can be built or tested end-to-end until a member can register and reach `approved_verified` status, since every community-facing feature requires an authenticated handle.

Contents

---

1. Scope

In scope: applicant registration, automated register lookup (HCPC/GMC/BASRAT/SST), automated identity document check (Onfido), the resulting status state machine, the admin manual-review fallback, and the point at which a verified member is handed off to authentication (EPIC-B mints the handle itself — see Section 7).

Out of scope for this spec (tracked elsewhere): - Handle creation/profile — EPIC-B, once `approved_verified` is reached. - Periodic reverification — explicitly Phase 2 per PRD Section 8.1, Step 4. - Appeals process for

`rejected` decisions — not detailed in the PRD beyond “reason provided”; flagged as an open question (Section 11). - Admin panel UI framework — covered generically in the architecture spec, Section 7.2; this document only specifies the queue’s data and ordering requirements.

---

2. Data model

This epic owns the `identity.members`, `identity.verification_evidence`, and `identity.verification_decisions` tables defined in the architecture spec, Section 4.1. Reproduced here with the additions this spec requires:

```
identity.members
  id                    uuid PK
  legal_name            text
  email                 text unique
  professional_body     enum(hcpc, gmc, basrat, sst)
  registration_number   text
  registration_country  text                  -- 'UK' only for
MVP; column exists
                                                -- for the
international-expansion
                                                -- goal without
needing a migration
  verification_status   enum(pending, needs_more_info,
approved_verified,
                              rejected, suspended)
  status_updated_at     timestamptz
  created_at            timestamptz

  unique(professional_body, registration_number,
registration_country)
    -- prevents the same real-world registration being used to
create two accounts

identity.verification_evidence
  id            uuid PK
  member_id     uuid FK -> identity.members
  evidence_type enum(register_lookup, onfido_check,
manual_document)
  source        text
  raw_result    jsonb
  outcome       enum(pass, fail, needs_review)
  created_at    timestamptz

identity.verification_decisions        -- immutable: INSERT-only
grant
  id            uuid PK
  member_id     uuid FK -> identity.members
  from_status   text
  to_status     text
  decided_by    text            -- 'system' or an admin's
member_id
```

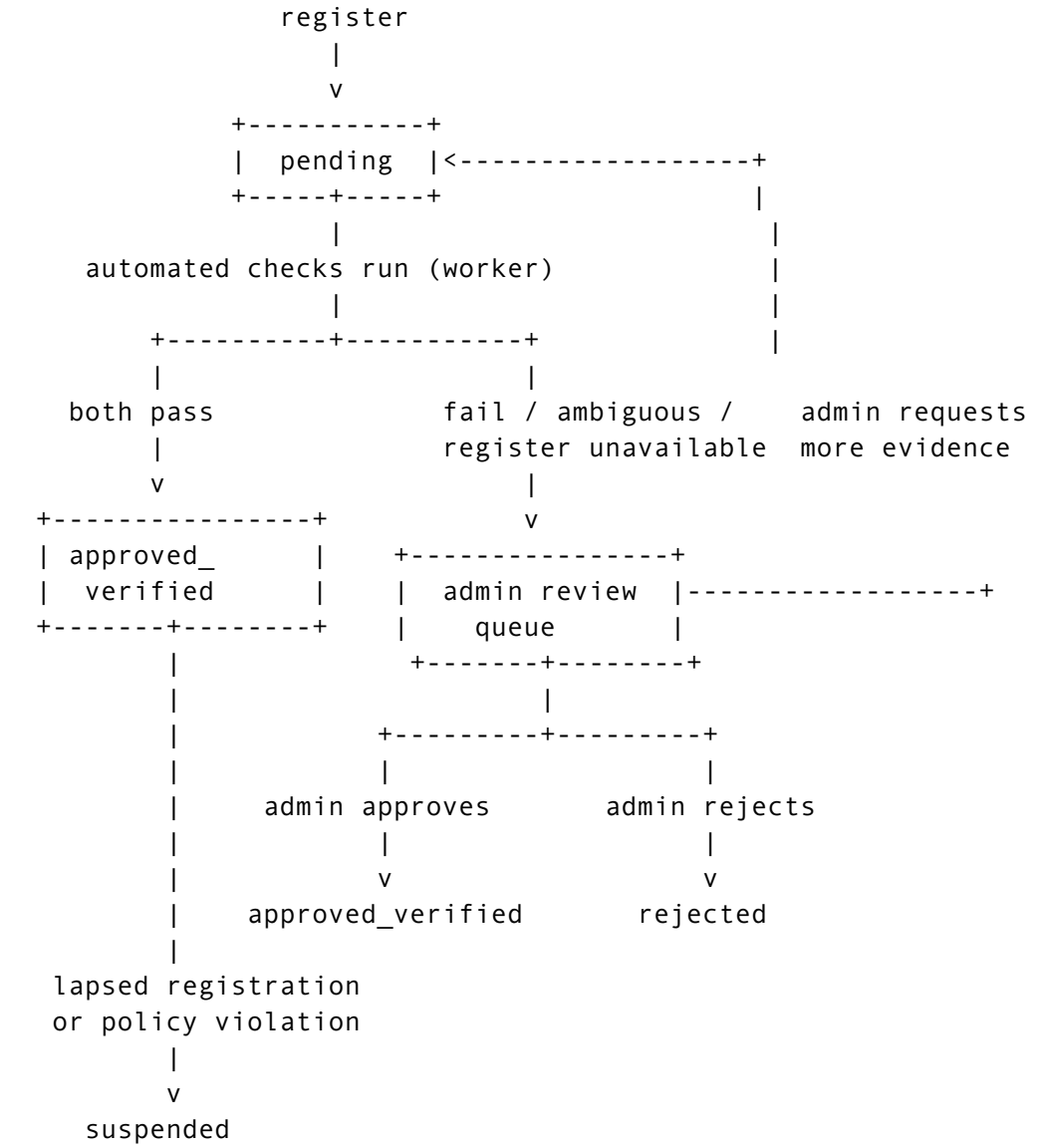
```
reason          text
created_at      timestamptz
```

Why a uniqueness constraint on `(professional_body, registration_number, registration_country)`: the PRD doesn't mention duplicate-registration abuse directly, but the trust proposition ("every member is a qualified professional") is undermined if one real registration can back multiple pseudonymous handles — that would let a single practitioner run sockpuppet accounts to manufacture apparent consensus. Enforced at the database level, not just application logic, consistent with the architecture spec's general approach of using schema constraints over code discipline.

No new table is needed for the admin review queue itself — it's a filtered view over `identity.members` (Section 6), not separate storage.

---

3. State machine



Rules: - Every transition is written as an `identity.verification_decisions` row (`from_status`, `to_status`, `decided_by`, `reason`) — no status field update happens without a corresponding decision row in the same database transaction. This is what makes the audit trail authoritative rather than incidental. - `needs_more_info` is a sub-state reachable only from the admin queue, not from the automated path directly — automated checks either resolve cleanly (both pass) or fall through to manual review; only a human admin decides that a specific applicant should be asked for more evidence rather than being outright rejected or escalated further. - `suspended` is reachable from `approved_verified` only, per PRD Section 8.1 Step 4 (lapsed registration or policy violation) — always admin/system-initiated, never self-service. - There is no path back to `pending` — `needs_more_info` is the only "still in progress" state after the initial automated pass, keeping the state machine's "still awaiting a human decision" states to one, not two overlapping ones.

---

4. API endpoints

Builds directly on the architecture spec, Section 5.1's registration flow and Section 5.3's API principles (versioned under `/v1`, no non-`IdentityService` endpoint ever returns `member_id` or `legal_name`).

```
POST /v1/auth/register
  body: { legal_name, email, professional_body,
registration_number, registration_country }
  -> 201, { member_id, verification_status: "pending" }
     (member_id here is only ever returned to the registering
applicant themselves,
      pre-handle, as a reference for polling their own status —
never surfaced to
      any other member)
  -> 409 if (professional_body, registration_number,
registration_country) already
     has a non-rejected member row
  -> enqueues the verification job (Section 5)

GET /v1/auth/verification-status
  auth: pending-scoped token (Section 7)
  -> { verification_status, status_updated_at,
needs_more_info_reason? }
     Powers the holding page — the only thing a non-
approved_verified session
     can call.

--- admin-only, moderator-role JWT claim required (architecture
spec, Section 5.3) ---

GET /v1/admin/verification-queue?
status=pending,needs_more_info&cursor=...
  -> paginated list, ordered per Section 6

GET /v1/admin/verification-queue/:member_id
  -> full detail: verification_evidence rows, prior decisions,
```

```
applicant-submitted
     fields. This is a legitimate IdentityService read (the admin
is doing the job
     verification exists for), not a moderation
identity_access_log event — see
     Section 9 for why this distinction matters.

POST /v1/admin/verification-queue/:member_id/decide
  body: { to_status: approved_verified | rejected |
needs_more_info, reason }
  -> writes identity.verification_decisions, updates
identity.members.verification_status
  -> on approved_verified: triggers EPIC-B handle-creation prompt
(Section 7)
  -> on rejected/needs_more_info: triggers a notification (EPIC-G)
with `reason`
```

---

5. Verification worker

Runs on the background-worker ECS service defined in the architecture spec (Section 3), driven by BullMQ. One job per registration, two sequential steps:

Step A — register lookup. Query the professional body's public register (HCPC and GMC both publish one; BASRAT and SST need confirmation of API vs. scrape-only access — see Section 11) by `registration_number`, fuzzy-match `legal_name` against the register's returned name. Write `identity.verification_evidence` with `evidence_type = register_lookup`. A fuzzy-match rather than exact-match is necessary because registers store legal names with formatting variance (middle names, punctuation) that shouldn't cause a false fail.

Step B — identity document check. Submit an Onfido check binding the applicant's uploaded document + selfie to the claimed identity. This step is what proves the *person registering* is the person named on the register — Step A alone only proves the registration number is real. Onfido is async; the job waits on a webhook rather than polling, per the architecture spec's worker-handles-anything-that-shouldn't-block-a-request principle. Result written as `evidence_type = onfido_check`.

Decision logic (executed once both results are in, or on evidence of definitive failure from either):

Register lookup	Onfido check	Outcome
pass	clear	auto-approve → `approved_verified`
fail	(any)	→ admin review queue, stays `pending`
needs_review / register unavailable	(any)	→ admin review queue, stays `pending`
pass	needs_review / fail	→ admin review queue, stays `pending`

Auto-approval is the only decision this worker makes unattended; every other outcome is a queue entry, not an auto-reject — a false auto-reject would permanently and wrongly block a real practitioner, whereas a false queue entry only costs admin time. This is a deliberate asymmetry: the automated path is only trusted to say “yes, cleanly,” never “no.”

---

6. Admin review queue

A view over `identity.members` where `verification_status IN (pending, needs_more_info)`, filtered to rows where the automated worker has already run (i.e., excluding the brief pending window while the worker job is still in flight) — this keeps the queue showing applicants who genuinely need a human, not ones still mid-automated-check.

Ordering: oldest `status_updated_at` first within each status, `needs_more_info` entries before fresh `pending` entries that fell through to review — an applicant who has already responded to an admin’s request for more evidence shouldn’t wait behind a fresh queue entry. (This queue has no `identifiable_patient_information`-style priority category — that ordering rule from the architecture spec, Section 7.2, is specific to the moderation-reports queue, not this one.)

Admin actions available per entry: approve (→ `approved_verified`), reject (→ `rejected`, `reason` required), request more info (→ `needs_more_info`, `reason` required — surfaced to the applicant so they know what to provide).

Verification is founder-led at MVP scale (PRD Section 8.3) — this queue is designed for a handful of admins reviewing a low volume of edge cases per day, not a high-throughput ops tool. A dedicated verification-operations function, if the platform scales, is a staffing decision, not something this spec needs to design ahead of.

---

7. Authentication handoff

EPIC-A ends at `approved_verified`; EPIC-B (handles/profile) begins immediately after. The precise handoff, since both epics touch the same registration flow:

1. On `POST /v1/auth/register`, no handle exists yet and no session is issued — the applicant only has a `member_id` to poll their own status with (Section 4).
2. While `verification_status` is anything other than `approved_verified`, any authenticated call is scoped to a **pending token** that grants access only to `GET /v1/auth/verification-status` — matching the PRD’s “all other statuses see a holding page only” and the architecture spec’s Section 5.2 note that non-verified members get a holding-status-scoped token.
3. The moment `verification_status` transitions to `approved_verified` (whether by auto-approve or admin decision), the member is prompted to choose a handle. Only once `community.handles` has a row for them does `IdentityService` issue the full handle-scoped JWT described in

the architecture spec, Section 5.2 — EPIC-A never issues a handle-scoped token itself, since it has no concept of handles.

This spec does not define the handle-creation endpoint itself (`community.handles` insert, uniqueness/profanity checks on `handle_name`) — that's EPIC-B's spec. It only guarantees the trigger point and the shape of what EPIC-B receives (`member_id`, resolved server-side, never exposed to the applicant's own client beyond the initial registration response).

---

8. Failure modes and edge cases

- **Register API down or rate-limited**: worker marks the `register_lookup` evidence row `outcome = needs_review` with the raw error in `raw_result`, routes to admin queue rather than retrying indefinitely and leaving the applicant in limbo. PRD's risk register (Section 13) already names this as a medium-likelihood risk with manual review as the designed mitigation.
- **BASRAT/SST have no confirmed public lookup API** (see Section 11) — until confirmed, these two professional bodies route every applicant straight to manual review regardless of Onfido outcome. This is a deliberate, explicit fallback, not a silent gap.
- **Onfido webhook never arrives** (delivery failure, applicant abandons the document-upload step): job has a timeout (proposed: 72 hours) after which it's surfaced in the admin queue as `needs_more_info` with an automatic reason of "identity check not completed," rather than leaving the applicant in `pending` indefinitely with no visible next step.
- **Duplicate registration attempt** on an already-`rejected` member: the unique constraint in Section 2 only blocks non-rejected rows, so a genuinely rejected applicant isn't permanently locked out of ever reapplying (e.g. after resolving a registration lapse) — a fresh registration attempt creates a new `identity.members` row. Whether this needs a cooldown period or admin awareness of the prior rejection is an open question (Section 11).
- **Applicant emails don't match register-held contact details**: not a verification signal — the register lookup matches on `legal_name` + `registration_number`, not email, since professional registers don't reliably expose a queryable email field.

---

9. Non-functional notes specific to EPIC-A

- **Why admin queue reads aren't** `identity_access_log` **events**: the architecture spec, Section 4.4, draws the line at "routine automated system access" vs. moderator-initiated identity access. Reviewing a *pending verification applicant's own submitted evidence* is the core, ordinary function `IdentityService` exists to perform — it is meaningfully different from a moderator looking up an *existing verified member's* real identity behind their handle (which the PRD's Section 9.4 access rule and the architecture spec's `identity_access_log` are protecting against). Conflating the two would make routine verification work indistinguishable from a genuine identity-access event in the audit trail, undermining the log's value as a trust artifact. If this reading is wrong, it needs correcting before build, not after — flagged in Section 11.
- **PII handling in transit/rest**: `legal_name`, `raw_result` (which may embed applicant-submitted document data from Onfido), and `registration_number` are exactly the category of data the architecture

spec's `identity` schema access grants exist to protect — no change needed here beyond confirming this epic's code paths only ever run under the `IdentityService` database role.
- **Rate limiting**: `POST /v1/auth/register` and `POST /v1/auth/request-link` are the two endpoints explicitly called out in the architecture spec, Section 5.3, for Redis-backed per-IP rate limiting — registration is an obvious target for automated abuse (e.g. scripted submission of stolen registration numbers to probe the register-lookup step).

---

10. Test plan

- **State machine**: every transition in Section 3 has a corresponding test; explicitly test that no transition exists that skips writing a `verification_decisions` row (e.g. by asserting the two writes happen in one database transaction and a forced failure of either rolls back both).
- **Uniqueness constraint**: registering the same `(professional_body, registration_number, registration_country)` twice while the first is `pending`/`approved_verified`/`needs_more_info`/`suspended` is rejected; while `rejected`, a new row is allowed.
- **Worker decision table**: each row of the Section 5 table covered by a test with mocked register-lookup and Onfido responses, including the "register unavailable" case.
- **Access-control regression** (extends the architecture spec's Section 9 cross-schema test): confirm no EPIC-A endpoint response DTO includes `legal_name` or raw `identity.members` fields beyond `member_id` and `verification_status`.
- **Timeout handling**: simulate an Onfido webhook that never arrives and assert the applicant surfaces in the admin queue after the timeout window, not stuck silently in `pending`.

---

11. Open questions

- **BASRAT/SST register access**: confirm whether either publishes a queryable API or only a manual/scrape-only public register. This determines whether Step A (Section 5) can run automatically for these two bodies at all, or whether every BASRAT/SST applicant is manual-review-only from day one. Relevant to FD-1 (professional scope) but is itself an implementation detail, not a scope decision.
- **Rejection cooldown/reapplication**: should a `rejected` applicant be able to immediately resubmit (current design permits this via Section 2's constraint scoping), or should there be a cooldown, or should the admin reviewing a resubmission see the prior rejection reason? Not addressed in the PRD.
- **Appeals process**: PRD Section 8.1 says a rejection reason is provided but doesn't describe an appeals mechanism beyond reapplying from scratch. Worth confirming with Andrew Renshaw/Paul Gouge whether MVP needs anything more formal.
- **Onfido webhook timeout value**: 72 hours is this spec's proposal, not a PRD-specified figure — worth a sanity check against Onfido's own typical turnaround time before committing to it.
- **Identity-access-log boundary (Section 9)**: confirm the reasoning that routine admin verification-queue review is not an `identity_access_log`

event is correct before build — it's a meaningful interpretation of PRD Section 9.4, not an explicit PRD statement.